

Lab SHEET 05

(FULL STACK)

PRIYANSHU SHARMA ,CU 240250980 (47) ,SEC(B)

1. Install packages

npm init -y

npm install express dotenv

npm install --save-dev nodemon

Terminal Output

```
PS D:\express-lab> npm init -y
PS D:\express-lab> npm install express
PS D:\express-lab> npm install --save-dev nodemon
```

Server running at http://localhost:4000

Browser Output

GET http://localhost:4000/

Express Lab Running

// PART A - PROJECT SETUP

```
require("dotenv").config();
```

```
const express = require("express");
```

```
const path = require("path");
```

```
const app = express();
```

```
const PORT = process.env.PORT || 4000;
```

GET /about

```
GET http://localhost:4000/about
{
  "name": "Priyanshu Sharma",
  "rollNo": "CU240250980"
}
```

GET /courses

```
GET http://localhost:4000/courses
[
  "Full Stack Web Development",
  "Data Structures",
  "Machine Learning"
]
```

POST /echo

```
POST http://localhost:4000/echo
Request: { "name": "Priyanshu", "course": "CSE" }
Response: { "name": "Priyanshu", "course": "CSE" }
```

GET /students/:id

```
GET http://localhost:4000/students/101
{
  "id": "101",
  "message": "Student details"
}
```

GET /search

```
GET /search?name=Priyanshu&age=21
{
  "name": "Priyanshu",
  "age": "21"
}
```

GET /products/:category/:id

```
GET /products/mobile/101
{
  "category": "mobile",
  "id": "101"
}
```

// PART D - CUSTOM LOGGER MIDDLEWARE

```
app.use((req, res, next) => {
```

```
console.log(
```

```
`${req.method} ${req.originalUrl} - ${new Date().toISOString()}`
```

Logger Output

```
GET /about 2026-09-16T07:52:00.000Z  
GET /courses 2026-09-16T07:51:10.000Z  
GET /admin/dashboard 2026-09-16T07:52:20.000Z
```

Without API Key

```
GET /admin/dashboard  
Status: 403 Forbidden  
{  
  "error": "Forbidden"  
}
```

With x-api-key

```
Header: x-api-key = 12345  
Status: 200 OK  
{  
  "message": "Welcome to Admin Dashboard",  
  "data": "Protected data"  
}
```

```
next());
```

```
// PART E - JSON AND FORM MIDDLEWARE
```

```
app.use(express.json());
```

```
app.use(  
  express.urlencoded({  
    extended: true}
```

```
// PART I - STATIC FILES
```

```
app.use(express.static(path.join(__dirname, "public")));
```

```
// PART A - BASIC SERVER
```

```
app.get("/", (req, res) => {  
  res.send("Express Lab Running");});
```

```
// PART B - BASIC ROUTING
```

```
app.get("/about", (req, res) => {  
  res.json({
```

```
    name: "Priyanshu Sharma",
```

```
    rollNo: "CU240250980"});
```

```
app.get("/courses", (req, res) => {
```

```
  res.json([
```

```
    "Full Stack Web Development",
```

```
    "Data Structures",
    "Machine Learning"]]);
app.post("/echo", (req, res) => {
  res.json(req.body));
// PART C - ROUTE PARAMETERS AND QUERY STRINGS
app.get("/students/:id", (req, res) => {
  res.json({
    id: req.params.id,
    message: "Student details" });
app.get("/search", (req, res) => {
  res.json({
    name: req.query.name,
    age: req.query.age});
app.get("/products/:category/:id", (req, res) => {
  res.json({
    category: req.params.category,
    id: req.params.id
// PART E - REGISTER AND CONTACT
app.post("/register", (req, res) => {
  const {
    name,
    email,
    password
  } = req.body;
  res.json({
    message: `Registration successful for ${name}`,
```

```

email: email,
passwordReceived: Boolean(password)  });
app.post("/contact", (req, res) => {
console.log("Contact Form Data:");
console.log(req.body);
res.json({
message: "Thank you for contacting us",

```

```

POST /register
POST http://localhost:4000/register
Request: { "name": "Priyanshu", "email": "priyanshu@gmail.com", "password": "123456" }
Response:
{
  "message": "Registration successful for Priyanshu",
  "email": "priyanshu@gmail.com"
}

POST /contact
Form data: name=Priyanshu, email=priyanshu@gmail.com, message=Hello
Console: Contact Form Data received
Response: { "message": "Thank you for contacting us" }

```

```

data: req.body  });
// PART F - STUDENTS ROUTES
let students = [{
  id: 1,
  name: "Priyanshu Sharma",
  course: "CSE">{
  id: 2,
  name: "Rahul Kumar",
  course: "BCA">{
  id: 3,
  name: "Aman Singh",
  course: "CSE"}];
// PART F - BOOKS ROUTES
let books = [

```

```

{ id: 1,
  title: "JavaScript Basics",
  author: "John Smith"},
{ id: 2,
  title: "Learning Node.js",
  author: "Jane Doe"]];
app.get("/api/books", (req, res) => {
  res.json(books));});
if (!book) {
  const error = new Error("Book not found");
  error.status = 404;
  return next(error);
res.json(book)
});
id: books.length
  ? Math.max(...books.map(b => b.id)) + 1
  title: req.body.title
  author: req.body.author  };
  books.push(newBook);
res.status(201).json(newBook));});
return next(error);  }
book.title = req.body.title ?? book.title;
book.author = req.body.author ?? book.author;
res.json(book);

```

Students Router

```
GET /api/students
[
  { "id": 1, "name": "Priyanshu Sharma", "course": "CSE" },
  { "id": 2, "name": "Rahul Kumar", "course": "BCA" }
]
```

Books Router

```
GET /api/books
[
  { "id": 1, "title": "JavaScript Basics", "author": "John Smith" },
  { "id": 2, "title": "Learning Node.js", "author": "Jane Doe" }
]
```

```
POST /api/books → 201 Created
```

```
});
```

```
// PART G - TASKS CRUD API
```

```
let tasks = [
```

```
{
```

```
  id: 1,
```

```
  title: "Complete Express Lab",
```

```
  completed: false
```

```
},
```

```
  id: 2,
```

```
  title: "Practice REST API",
```

```
  completed: truee];
```

```
app.get("/api/tasks", (req, res) => {
```

```
  res.json(tasks);});
```

```
if (!task) {
```

GET All

```
GET /api/tasks
[
  { "id": 1, "title": "Complete Express Lab", "completed": false },
  { "id": 2, "title": "Practice REST API", "completed": true }
]
```

POST

```
POST /api/tasks
Request: { "title": "Study Express", "completed": false }
Status: 201 Created
Response: { "id": 3, "title": "Study Express", "completed": false }
```

PUT / DELETE

```
PUT /api/tasks/2 → 200 OK → updated task
DELETE /api/tasks/1 → 200 OK → { "message": "Task deleted" }
```

```
// 5. DELETE TASK
```

```

app.delete("/api/tasks/:id", (req, res, next) =>
  const index = tasks.findIndex(
    t => t.id === Number(req.params.id) );
  if (index === -1) {
    const error = new Error("Task not found");
    error.status = 404;
    return next(error); }
  const deletedTask = tasks.splice(index, 1)[0];
  res.json({
    message: "Task deleted",

```

Static File Output

```

GET http://localhost:4000/

Welcome to Express Lab
Express Static Files Working

```

.env Configuration

```

PORT=4000

```

```

Terminal:
Server running at http://localhost:4000

```

```

task: deletedTask});

```

// PART J - LIBRARY MANAGEMENT MINI API

```

let members = [ {
  id: 1,
  name: "Priyanshu Sharma",
  email: "priyanshu@gmail.com"},
{ id: 2,
  name: "Rahul Kumar",
  email: "rahul@gmail.com"}];

// DELETE member
app.delete("/api/members/:id", (req, res, next) => {
  const index = members.findIndex(
    m => m.id === Number(req.params.id) );

```

```

if (index === -1) {
  const error = new Error("Member not found");
  error.status = 40
  return next(error);}
const deletedMember = members.splice(index, 1)[0];
res.json({
  message: "Member deleted",
    member: deletedMember});
// PART H - 404 ERROR HANDLER
app.use((req, res) => {
  res.status(404).json({
    error: "Route not found"});
// PART H - CENTRALIZED ERROR HANDLER
app.use((err, req, res, next) => {
  console.error("ERROR:", err.message);
  res.status(err.status || 500).json({
    error: err.message || "Internal Server Error"  });

```

Books CRUD

```

GET    /api/books      → 200 OK
GET    /api/books/1  → 200 OK
POST   /api/books      → 201 Created
PUT    /api/books/1    → 200 OK
DELETE /api/books/1    → 200 OK

```

Members CRUD

```

GET    /api/members  → 200 OK
GET    /api/members/1 → 200 OK
POST   /api/members  → 201 Created
PUT    /api/members/1 → 200 OK
DELETE /api/members/1 → 200 OK

```

Final Status

```

Request Logger: WORKING
Fake Authentication: WORKING
Routers: WORKING
404 Handler: WORKING

```

